Knowledge Pack — everything on the slides, explained from scratch

For the team. Read your slides' sections before the review. Nothing here assumes you have touched the code. If you can explain the section, you can present the slide and answer follow-ups.

`QA-PREP.pdf` is the companion: the questions the panel is likely to ask, with answers.

Part 0 — The project in six sentences

Computers write a line of text every time something happens: a login, a failed password, a program starting. These files are called **logs**. Companies buy a tool called a **SIEM** that reads those lines and raises an alert when a line matches a written rule.

Our project reads the same logs but judges lines by their **meaning** rather than their exact words, groups related alerts into one **incident**, labels the incident with the industry-standard attack code (**MITRE ATT&CK**), and has a small **language model running on the laptop itself** write a plain-English explanation of what happened.

The part that is unusual is not the detector. It is that we **measure how much detection each system loses when the attacker is deliberately hiding**, against a real production SIEM, on the same logs, at the same moment — and we report the cases where we lose.

Part 1 — Words you must be able to define

Word	What it means, plainly
Log	A text file where a computer records events, one line each. `Failed password for alice from 172.16.5.191` is a log line.
SIEM	Security Information and Event Management. The industry tool that collects logs and applies written rules to them. Splunk, QRadar, Wazuh.
Wazuh	A free, open-source, production SIEM. We run the real thing in a container and compare against it. We did not write its rules and cannot change them.
Rule / signature	A written pattern, e.g. "8 failed passwords from one IP in 120 seconds → alert". It matches exact text.
Embedding	A way of turning a sentence into a list of numbers (384 of them here) such that sentences with similar meaning get similar numbers. This is what lets us compare *meaning* instead of *spelling*.
MiniLM	The specific small model that produces those numbers (`all-MiniLM-L6-v2`). Tiny, fast, runs on a CPU.
Isolation Forest	A classic machine-learning method that finds the odd ones out in a pile of data, without being told in advance what "odd" looks like. No labelled examples needed.

Word	What it means, plainly
Per-user model	We build one Isolation Forest for each user, so "unusual" means unusual *for that person*, not for the company.
Incident	A group of related suspicious events — same user, close together in time — presented as one thing to the analyst instead of forty separate alerts.
MITRE ATT&CK	A public catalogue of attacker techniques, each with a code. `T1110` is Brute Force. Security teams speak in these codes.
LLM / language model	The thing that writes the English explanation. We use **phi3**, a small model that runs on the laptop with **Ollama**. It never decides whether something is an attack.
Camouflage	Changing an attack so it is harder to spot *without changing what it does*. Rewording the message, slowing it down, spreading it across several IP addresses.
Instance detection	Did the system notice the attack **at all**? One flagged line out of twenty counts as detected. This is the number our main result is measured in.
Line detection rate	What *share* of the attack's lines the system flagged. Finer-grained than instance detection.
Differential	How much more the SIEM's detection dropped than ours did, in percentage points. Our headline number, +31.1, is a differential.
Precision	Of everything we flagged, how much was really an attack. Ours is 0.29 — our known weakness.
Recall	Of all the real attack lines, how many we flagged. Ours is 0.86.
Seed	A number that fixes the random choices, so a run can be repeated exactly. We pool results over 5 seeds because one run is not evidence.

Part 2 — Slide by slide

Slides 3 and 4 — Background

What the slides say. Logs are enormous, so tools read them for us. Those tools match text against rules. Text matching breaks when the text changes.

Why slide 4 exists. The panel asked for proof rather than claims. Every row on that slide is something we ran ourselves:

- **Sigma, 0 of ~3,300.** Sigma is the community's open library of detection rules. We searched every rule in it for the four phrases that make up almost all Linux SSH security logging — `Failed password`, `Accepted password`, `Invalid user`, `pam_unix`. Not one rule matches. So for this attack surface the open rule library gives no coverage at all before any attacker does anything clever.
- **91% → 29%.** We fired a brute-force attack at Wazuh and it flagged 91% of the attack's lines. We fired the *same attack with the wording changed* and it flagged 29%. The attack did not get smarter; the words changed.
- **97-100%.** Our detector flagged nearly every line in both cases, because rewording barely moves the meaning, and meaning is what we compare.

- **4,546 vs 1,415.** On one clean corpus of real logs with 9 known attacks, Wazuh raised 4,546 alerts and we raised 1,415 incidents. Both numbers are far too high for 9 real attacks. That is alert fatigue, measured on both sides — including ours.

Say it like this: "We did not read that rules are brittle, we measured it. Wazuh keeps 91% of the attack when we play fair and 29% when we reword it."

Slide 5 — Problem statement

Two sentences, and both matter. The first is the technical problem (text matching misses meaning). The second is the *research* problem (nobody publishes how much is lost when the attacker actively hides). The second is what makes this a project and not a product.

Slide 6 — Motivation

Three reasons, in the order they matter: analysts are drowning in alerts; behaviour survives rewording where words do not; and the whole thing runs on a normal laptop, so a school, a hospital or a small company with no security budget could actually use it.

The laptop point is not a limitation we are excusing. It is a design decision, and it is why we refuse to make the GPU compulsory.

Slides 7 and 8 — Literature review

Eight journal papers, all from IEEE Transactions or Web of Science / Scopus indexed journals. The PDFs travel with the presentation.

The point to land: **every one of them evaluates on data where the attacker is not trying to evade the detector being tested.** One paper (R8) does have an active attacker, but it is about network traffic, not logs, and it does not compare with a SIEM. That is the hole we are working in.

If the panel asks about **DeepLog** or **LogBERT** — the famous names that used to be on this slide — the answer is: they are conference papers (ACM CCS 2017 and IJCNN 2021), and the panel asked for indexed journals, so they were replaced with journal work that covers the same ground. We know them and can discuss them; they are simply not journal articles.

Slide 9 — Research gaps

Three gaps, each pointing at a paper, ending on ours. Keep the last line memorised:

> How much worse does each system get when the attacker tries to hide — measured, on the same logs, at the same moment, for a research detector and a production SIEM together.

Slide 10 — Objectives

Six flat bullets. The one that is easy to skip and shouldn't be is the fifth: measuring against Wazuh under camouflage **including where we lose**. That sentence is the project.

Slide 11 — Proposed methodology (the diagram)

Read the diagram out loud in order:

1. **Read the logs.** Real SSH logs from a public research dataset (LogHub OpenSSH, 655,147 lines, which contains genuine brute-force attacks from the open internet), plus a small 751-line synthetic file where we know every answer and use it to check nothing broke.
2. **Parse and mask.** Split each line into fields, and replace the IP address, port and process ID with a symbol. Why: `Failed password from 10.0.0.5` and `Failed password from 10.0.0.9` are the same *kind* of event and should look identical to the model.
3. **Embed the meaning.** MiniLM turns the masked line into 384 numbers.
4. **Score each user.** An Isolation Forest per user flags the lines that do not fit that person's normal pattern. No labelled attack data is needed, which matters because no real company has labelled logs.
5. **Group into incidents.** Suspicious events from one user within 30 minutes become one incident. There is also a physics rule: the same account logging in from two places too far apart to travel between in the time available is impossible, so it is flagged regardless of what the embeddings think.
6. **Attach the MITRE technique** from a lookup table.
7. **Explain in English** with the local LLM.
8. **Show the analyst**, ranked.

The band along the bottom is the adversarial evaluation, and the arrow going back into step 1 is the point: camouflaged attacks are fed back in and both systems are re-scored every time anything changes.

The line on the slide that protects you: Isolation Forest over embeddings is standard and we say so. Claiming it as novel would be dishonest and the panel would catch it.

Slide 12 — System architecture (the diagram)

Three columns and a band.

- **Live sources.** Three ways lines arrive: tailing a file the way `tail -f` does; reading the machine's own systemd journal; or replaying a recording at its original speed for a demo.
- **Detection engine.** Parser → streaming processor → embedding and Isolation Forest → incidents and MITRE. The streaming processor scores in small batches (every 50 lines or 5 seconds, whichever comes first) and remembers where it got to, so a restart neither skips nor repeats lines.
- **What the analyst uses.** An HTTP service that other machines can read, and the dashboard on top of it. The LLM explanation is requested per incident, by clicking, because writing one takes 5 seconds on a GPU and 24 on a CPU — narrating all 94 as they arrive would put the detector minutes behind the log.
- **Comparison track.** Wazuh and the scorer live in a *separate package that the pipeline is not allowed to import*. There is an automated test that fails if anybody breaks that rule. Reason: if the thing being measured can reach into the thing doing the measuring, the measurement stops meaning anything.

One fact worth quoting: streaming and batch produce **identical** results on the same file — 751 lines, 130 flagged events, 94 incidents — and a test fails if that ever changes. That is what makes the live demo's numbers comparable to the recorded ones.

Saved detector model. We fit the model once on history and then only *use* it on the stream. It stores a fingerprint of which embedding model produced its training data and refuses to load against a different one — because otherwise it would happily score the wrong kind of numbers and return results that look completely normal and mean nothing.

Slide 13 — Implementation (the bake-off diagram)

The fairness problem: if we ran the attack twice, once for each system, the two systems would not have seen the same traffic and any difference could be ours to arrange.

The fix: **one writer**. Each line is appended to the file our detector is reading and to the file Wazuh is reading *in the same function call*, before the next line is generated.

The ground-truth problem: to score a system you must know which lines were the attack. Because our injector generated them, we record every line's number and label as it is written. So scoring is automatic, and the same scoring code runs on both systems' output.

That diagram is the answer to "how do we know you did not rig this?"

Slide 14 — Results

The table. Each number is how many percentage points of *instance detection* the system lost when the attack was camouflaged. Zero means the camouflage did nothing.

- `rephrase` — we lose 2.2, Wazuh loses 33.3. Difference **+31.1 to us**.
- Four strategies are ties at 0.0 for both.
- `ip_rotate`, `slow`, `realistic-combo` — **we lose and Wazuh does not**.

Why we win on rewording. Wazuh's SSH rules match literal strings. Change the wording and the line falls out of its decoder entirely. Our embedding of a reworded line stays close to the original in the number space, so it still looks like the same kind of event.

Why we lose on IP rotation. Our model builds a picture per user and leans on the context of the source; spreading an attack across many IPs breaks that. Wazuh's rules count per IP and simply fire on whichever IP still crosses the threshold.

Why we lose when it is slowed down. Spreading an attack over hours makes each individual line look ordinary to us. Wazuh, replayed at true speed, does not drop at all — though note that Wazuh firing a "brute force" rule on events spread over 15 hours is arguably indiscriminate rather than robust, which is why we report its alert volume too.

The threshold. Before writing any of this code, we wrote down in the project's git history: if the best difference is 25 points or more, continue; 10-25, do only the cheap work; under 10, stop and publish the negative result. We hit 31.1. Writing the rule down in advance is what stops us from moving the goalposts after seeing results.

The live number. On the live grid, rewording an impossible-travel attack took Wazuh from 3 of 3 instances to 0 of 3 while we stayed at 3 of 3. That is the biggest gap on the board and it happens because impossible travel is only two lines long — reword both and there is nothing left for a string-matching rule to catch. A 20-line brute force always leaves one line behind, which is why most cells tie.

The weaknesses box. Do not skip it. Precision 0.29 means most of what we flag is not an attack: 94 incidents raised where 9 were real. Impossible travel is caught by the physics rule, not the embeddings — so never say "our embeddings catch impossible travel". Both of these are written in our own

documentation as known issues, and the fix (modelling sequences of events rather than single lines) is the next stage.

Slide 15 — Working screenshots

All four are captures of the system running, taken for this deck. Numbers to quote:

- Replaying the test corpus: **751 lines seen, 751 parsed, 94 incidents, 19 notable**.
- The quiet baseline with no attack at all: we flagged **5 of 30** benign lines, Wazuh flagged **19 of 30**. Every alert in that run is a false positive by construction — it is the fairest picture of the noise problem on both sides.

Slide 16 — Technologies

Note three deliberate choices in case they are questioned:

- **No fine-tuning.** We do not train the language model. Training it would not fix the detection problem and would make the results much harder for anyone to reproduce.
- **Wazuh, not Splunk.** Splunk is commercial and cannot be scripted into an automated comparison for free. Wazuh is a real production SIEM, free, and runs the same rules a real deployment would.
- **CPU is the supported floor.** The GPU makes the explanation 4-5× faster. It is never required.

Slide 17 — Project schedule

Ticks stop at month 2 because that is what is genuinely finished. Months 3 and 4 are testing and documentation and have not been done.

Part 3 — Material that did not fit on a slide

The eight camouflage strategies

Strategy	What it does to the attack
`rephrase`	Says the same thing in different words
`mimicry`	Pads the attack with ordinary-looking lines so it blends in
`noise`	Adds unrelated junk events around it
`volume`	Changes how many events the attack produces
`ip_rotate`	Spreads the attack across many source addresses
`slow`	Stretches the same attack over hours instead of minutes
`temporal`	Moves the attack to a different time of day
`realistic-combo`	Several of the above together — the closest to a real attacker

Why the timing strategies need a different instrument

Wazuh decides "8 failures in 120 seconds" using the time the line **arrives at it**, not the timestamp written in the line. We verified this: 15 identical lines fire the same rule whether we feed them 3

seconds apart or an hour apart. So if we replay a log file at full speed, every event arrives at once and Wazuh's score under a "slow" attack is fixed by our replay speed rather than by its rules.

That is why those three strategies are measured by a **paced replay** that writes lines at their real spacing, which took 6.5 hours to run. Fixing the instrument is not the same as changing the criterion: the threshold, the strategies and the deciding quantity were all left exactly as registered.

Why not HDFS or BGL

Those are the datasets the literature uses, and they are labelled, which is convenient. We rejected them because their "anomalies" are **hardware and software faults, not attackers**. With no attacker there is no MITRE technique to attach, no attack story to explain, and camouflage is meaningless — you cannot ask a failing disk to hide. Using them would have benchmarked the one layer we do not claim as our contribution while switching off both of the things that make this project distinctive.

How the real corpus was built

OpenSSH's logs turned out to be **66-83% attack traffic** — the open internet hammers any SSH server continuously. If we had injected our attacks straight in, brute force would have been the statistical *normal*, and our attack would score as ordinary. So the background is built explicitly: real benign lines, real usernames, the real host, plus real attacker traffic thinned to a declared percentage that we then report results across. Everything in the background is real; what is authored is the injected attack and the camouflage.

Numbers, and where they come from

Number	Source file in the repository
+31.1 / four ties / −11.1, −8.9, −13.3	`results/killpoint/killpoint_eight_strategies_ambient0.05.md`
4,546 Wazuh alerts vs 1,415 our incidents	`results/killpoint/killpoint_ambient0.05.md`
Wazuh 91% → 29% of brute-force lines	`results/bakeoff/bakeoff.md`
Impossible travel reworded: Wazuh 0/3, ours 3/3	`results/bakeoff/bakeoff.md`
Quiet baseline: ours 5/30, Wazuh 19/30	`results/bakeoff/act1_benign/benign_baseline.md`
Precision 0.29 / recall 0.86 / F1 0.43	`results/evaluation.md`
751 → 130 → 94, streaming equals batch	`tests/test_stream_regression.py`
3.9 s GPU vs 18.2 s CPU narration	measured 2026-09-01 on the demo machine (RTX 4060 / i7-13650HX), recorded in the project log